Ch-1 Pyton Program

(2) Data Types

Python
```
a = 10
b = 3.14
c = 2 + 3j
d = True
e = "Hello"

print("Integer:", a, type(a))
print("Float:", b, type(b))
print("Complex:", c, type(c))
print("Boolean:", d, type(d))
print("String:", e, type(e))
```

Output:

text
```
Integer: 10 <class 'int'>
Float: 3.14 <class 'float'>
Complex: (2+3j) <class 'complex'>
Boolean: True <class 'bool'>
String: Hello <class 'str'>
```

---

(3) id(), type(), range()

Python
```
x = 100
y = "Python"

print("Type of x:", type(x))
print("ID of x:", id(x))
print("range(5):", list(range(5)))
print("range(2,10,2):", list(range(2,10,2)))
```

Output:

text
```
Type of x: <class 'int'>
```

ID of x: 1407... (some number)
range(5): [0, 1, 2, 3, 4]
range(2,10,2): [2, 4, 6, 8]

---

(4) Type Conversion

Python

```
print(int("100"))
print(float("3.14"))
print(str(50))
print(int(7.8))
print(complex(5))
```

Output:

text

```
100
3.14
50
7
(5+0j)
```

---

(5) Operators

Python

```
print("Arithmetic:", 10+5, 10-5, 10*5, 10/5, 10//5, 10%3,
2**3)
print("Relational:", 10>5, 10==10, 10!=5)
print("Logical:", True and False, True or False)
print("Ternary:", "Adult" if 20>=18 else "Minor")
```

Output:

text

```
Arithmetic: 15 5 50 2.0 2 1 8
Relational: True True True
Logical: False True
Ternary: Adult
```

---

(6) input(), print(), sep, end, format

Python

```
name = "Amit"
age = 21

print("Name:", name, "Age:", age, sep=" | ", end=" ***\n")
print("Hello {0}, Age: {1}".format(name, age))
```

Output:

text

```
Name: | Amit | Age: | 21 ***
Hello Amit, Age: 21
```

---

(7) Conditional Statements

Python

```
num = 10

if num > 0:
    print("Positive")
elif num < 0:
    print("Negative")
else:
    print("Zero")

if num % 2 == 0:
    print("Even Number")
```

Output:

text

```
Positive
Even Number
```

---

(8) Loops

Python

```
print("While Loop:")
i = 1
while i <= 5:
    print(i, end=" ")
```

```
    i += 1

print("\nFor Loop:")
for i in range(1, 6):
    print(i, end=" ")
```

Output:

```
text
While Loop:
1 2 3 4 5
For Loop:
1 2 3 4 5
```

---

(9) break, continue, pass

```
Python
print("Break Example:")
for i in range(10):
    if i == 5:
        break
    print(i, end=" ")

print("\nContinue Example:")
for i in range(6):
    if i == 3:
        continue
    print(i, end=" ")
```

Output:

```
text
Break Example:
0 1 2 3 4
Continue Example:
0 1 2 4 5
```

---

(10) String Indexing & Slicing

```
Python
s = "Python"

print(s[0], s[1], s[-1])
```

```
print(s[0:4])
print(s[::2])
print(s[::-1])
```
Output:
```
text
P y n
Pyth
Pto
nohtyP
```

---

(11) File Read & Write
```
Python
# Write
with open("demo.txt", "w") as f:
    f.write("Hello Python\nFile Handling Demo.")

# Read
with open("demo.txt", "r") as f:
    print(f.read())
```
Output:
```
text
Hello Python
File Handling Demo.
```

---

(12) Copy File
```
Python
with open("demo.txt", "r") as src:
    with open("copy.txt", "w") as dest:
        dest.write(src.read())

print("File copied successfully!")
```
Output:
```
text
File copied successfully!
```

---

(13) Creating Lists

Python
```
lst1 = [1,2,3]
lst2 = list(range(5))
lst3 = [x*x for x in range(5)]

print(lst1)
print(lst2)
print(lst3)
```
Output:
text
```
[1, 2, 3]
[0, 1, 2, 3, 4]
[0, 1, 4, 9, 16]
```

---

(14) List Methods (All)
Python
```
nums = [10, 20, 30, 20, 40]

print(len(nums))
print(nums.count(20))
nums.append(50)
nums.insert(2, 99)
nums.sort()
print(nums)
```
Output:
text
```
5
2
[10, 20, 20, 30, 40, 50, 99]
```

---

(15) Creating Tuples
Python
```
t1 = (1, 2, 3)
t2 = 10, 20, 30
t3 = tuple([5, 6, 7])

print(t1)
```

```
print(t2)
print(t3)
```
Output:
```
text
(1, 2, 3)
(10, 20, 30)
(5, 6, 7)
```

---

(16) Patterns

Python
```
# Pattern 1
for i in range(1, 6):
    print(str(i) * i)

# Pattern 2 (Alphabets)
for i in range(5):
    for j in range(i+1):
        print(chr(65+j), end=" ")
    print()

# Star Pattern
for i in range(5, 0, -1):
    print('*' * i)
```
Output:
```
text
1
22
333
4444
55555
A
A B
A B C
A B C D
A B C D E
*****
****
```

***
**
*

---

(17) Tuple Methods
Python
```
t = (1, 2, 2, 3, 4, 5)

print(len(t))
print(t.count(2))
print(t.index(3))
print(max(t))
print(min(t))
print(sorted(t))
```
Output:
text
```
6
2
2
5
1
[1, 2, 2, 3, 4, 5]
```

---

(18) Creating Sets
Python
```
s1 = {1, 2, 3, 4}
s2 = set([10, 20, 30])
s3 = set("Python")

print(s1)
print(s2)
print(s3)
```
Output:
text
```
{1, 2, 3, 4}
{10, 20, 30}
{'P', 'y', 't', 'h', 'o', 'n'}
```

16 Stack Implementation (LIFO) – With `push`, `pop`, `peek`, `display`

```
            # Stack using List

class Stack:
    def __init__(self):
        self.stack = []

    # Push operation
    def push(self, item):
        self.stack.append(item)
        print(item, "pushed into stack")

    # Pop operation
    def pop(self):
        if len(self.stack) == 0:
            return "Stack is Empty"
        return self.stack.pop()

    # Peek operation
    def peek(self):
        if len(self.stack) == 0:
            return "Stack is Empty"
        return self.stack[-1]

    # Check if stack is empty
    def isEmpty(self):
        return len(self.stack) == 0

    # Size of stack
    def size(self):
        return len(self.stack)

    # Display stack
    def display(self):
        print("Stack:", self.stack)


# Driver Program
s = Stack()

s.push(10)
s.push(20)
s.push(30)

s.display()

print("Top Element:", s.peek())

print("Popped Element:", s.pop())
```

```
s.display()

print("Stack Size:", s.size())

print("Is Stack Empty?", s.isEmpty())
```

o/p
10 pushed into stack
20 pushed into stack
30 pushed into stack
Stack: [10, 20, 30]
Top Element: 30
Popped Element: 30
Stack: [10, 20]
Stack Size: 2
Is Stack Empty? False